SWEN3 · PAPERLESS · SPRINT 3

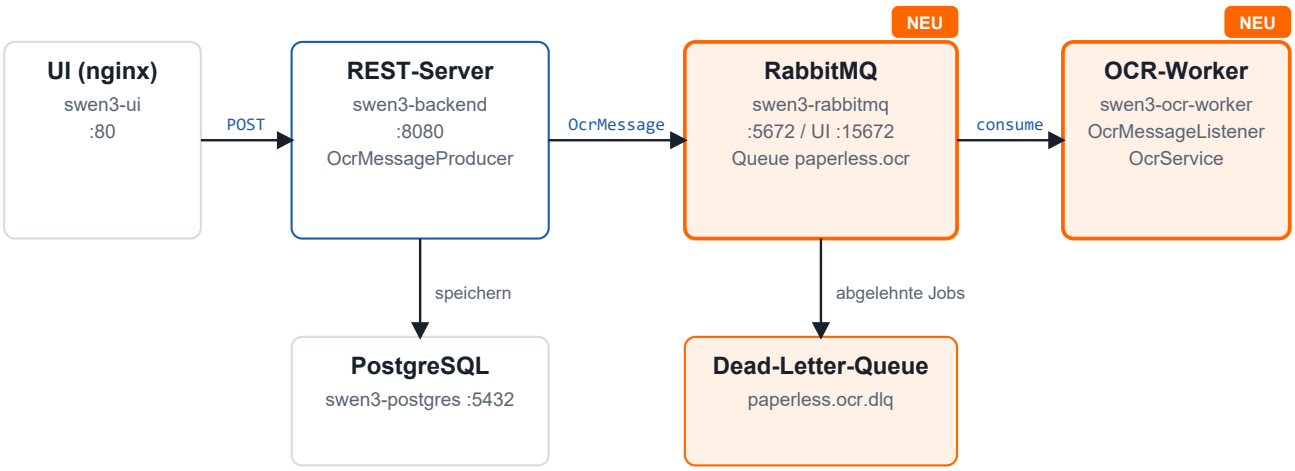
RabbitMQ & OCR-Worker

Was neu dazugekommen ist, wo du es im Projekt findest und wie es funktioniert.

Kurz gesagt: Wenn ein Dokument angelegt wird, speichert der REST-Server es in Postgres und schickt danach eine Nachricht (einen „OCR-Job“) an **RabbitMQ**. Ein neuer, eigener Dienst, der **OCR-Worker**, holt die Nachricht aus der Queue und verarbeitet sie. Im Moment prüft und loggt er sie nur; die echte Texterkennung kommt in einem späteren Sprint. Dazu gibt es eigene Exceptions pro Schicht und Logging an den wichtigen Stellen.

Branch: `feature/sprint3-rabbitmq`

Architektur



Orange = neu in Sprint 3. Der Browser spricht weiterhin nur mit nginx; der Worker hat keinen Port nach außen, er ist nur mit RabbitMQ verbunden.

Starten und ausprobieren

```
git fetch
git checkout feature/sprint3-rabbitmq
docker compose up -d --build
docker compose ps          # 5 Container: postgres, rabbitmq, backend, ocr-worker, ui
```

Was	Adresse
UI	`http://localhost`
Backend direkt	`http://localhost:8080/api/documents`
RabbitMQ Management-UI	`http://localhost:15672` · Login `paperless` / `paperless`
OCR-Worker (kein Port)	Logs live ansehen: `docker logs -f swen3-ocr-worker`

Ausprobieren: In der UI ein Dokument anlegen. Im Worker-Log erscheint kurz danach `Received OCR job ...` und `OCR job for document X finished`. In der Management-UI unter Queues sieht man die beiden Queues und einen verbundenen Consumer.
IntelliJ: Der Worker ist ein eigenes Maven-Projekt. Einmal `ocr-worker/pom.xml` → Rechtsklick → Add as Maven Project, sonst erkennt die IDE ihn nicht.

Die 5 Sprint-Punkte

1 RabbitMQ in docker-compose

Anforderung: Extend docker-compose.yml to run RabbitMQ in a container

WAS

RabbitMQ ist ein **Message Broker**: ein Briefkasten zwischen Diensten. Der REST-Server legt Nachrichten hinein, der Worker holt sie ab. Beide müssen sich nicht kennen und nicht gleichzeitig laufen.

WO

Datei	Inhalt
`docker-compose.yml`	Service `rabbitmq` (Image `rabbitmq:4-management`), Service `ocr-worker`; das Backend bekommt die RabbitMQ-Verbindungsdaten

WIE ES FUNKTIONIERT

- Ports: `5672` = AMQP (Protokoll für Backend und Worker), `15672` = Management-UI im Browser.
- Ein **Healthcheck** (`rabbitmq-diagnostics -q ping`) meldet, wann RabbitMQ bereit ist.
- Backend und Worker starten erst danach (`depends_on: condition: service_healthy`).
- Benutzer und Passwort kommen über `RABBITMQ_DEFAULT_USER` / `RABBITMQ_DEFAULT_PASS`; im Container wird der Host über `SPRING_RABBITMQ_HOST: rabbitmq` gesetzt.

2 Queues im REST-Server

Anforderung: Integrate Queues into REST Server

WAS

Der REST-Server kann jetzt mit RabbitMQ sprechen und legt beim Start die nötigen Queues an.

WO

Datei	Inhalt
`pom.xml`	neue Abhängigkeit `spring-boot-starter-amqp`
`config/RabbitMqConfig.java`	Queue `paperless.ocr` mit Dead-Letter-Queue `paperless.ocr.dlq`; JSON-Converter
`resources/application.properties`	`spring.rabbitmq.*` (Host, Port, Login, Timeout 5 s) und Queue-Namen `paperless.rabbitmq.*`
`test/resources/application.properties`	Queue-Namen für die Tests

WIE ES FUNKTIONIERT

- Die Queues sind `durable`: Sie überleben einen Neustart von RabbitMQ.
- **Dead-Letter-Queue:** Die Hauptqueue hat die Argumente `x-dead-letter-exchange` und `x-dead-letter-routing-key`. Lehnt der Worker eine Nachricht ab, verschiebt RabbitMQ sie automatisch in `paperless.ocr.dlq`, statt sie wegzuwerfen.
- Nachrichten werden als **JSON** verschickt (`JacksonJsonMessageConverter`), damit sie lesbar und unabhängig von Java-Klassen sind.
- Der Worker deklariert die Queues mit **identischen Argumenten**. Deshalb ist egal, wer zuerst startet. Wer etwas an der Queue-Konfiguration ändert, muss es in beiden Projekten ändern, sonst lehnt RabbitMQ die Deklaration ab (PRECONDITION_FAILED).

3 Nachricht beim Upload und leerer OCR-Worker

Anforderung: On document upload the REST-Server should also send a message to RabbitMQ, processed by an „empty“ OCR worker

WAS

Nach dem Speichern eines Dokuments geht ein OCR-Job an die Queue. Als „Upload“ zählt das bestehende `POST /api/documents`, weil es noch keinen echten Datei-Upload gibt.

WO

Datei	Inhalt
`service/DocumentService.java`	`create()` speichert und ruft danach den Producer auf
`messaging/OcrMessage.java`	die Nachricht: `documentId`, `filename`, `contentType`, `uploadedAt`
`messaging/OcrMessageProducer.java`	schickt die Nachricht mit `RabbitTemplate.convertAndSend()`
`ocr-worker/`	neuer Dienst: eigene Spring-Boot-App, eigenes Dockerfile, eigener Container
`ocr-worker/.../OcrMessageListener.java`	`@RabbitListener` empfängt die Jobs
`ocr-worker/.../OcrService.java`	prüft und loggt den Job (absichtlich leer, echte OCR folgt)

WIE ES FUNKTIONIERT

- Ablauf: UI → POST → DocumentService speichert in Postgres → OcrMessageProducer sendet → RabbitMQ → OcrMessageListener empfängt → OcrService verarbeitet.
- Der Worker hat seine eigene Kopie von `OcrMessage`. Weil die Nachricht JSON ist, zählt nur, dass die **Feldnamen** übereinstimmen, nicht das Java-Package.
- Verliert der Worker die Verbindung (z. B. RabbitMQ-Neustart), verbindet er sich automatisch wieder.
- Im OcrService steht ein `TODO`, wo später Datei laden und OCR ausführen hinkommt.

Beispiel einer Nachricht in der Queue

```
{
  "documentId": 6,
  "filename": "rechnung-10.pdf",
  "contentType": "application/pdf",
  "uploadedAt": "2026-10-18T11:33:24.36"
}
```

4 Fehlerbehandlung mit Exceptions pro Schicht

Anforderung: Failure/exception-handling (with layer-specific exceptions) implemented

WAS

Jede Schicht hat ihre eigene Exception. So sieht man sofort, wo ein Fehler entstanden ist, und jede Art von Fehler bekommt den passenden HTTP-Status. Alle Klassen liegen in `exception/`.

WO

Datei	Inhalt
`exception/ServiceException.java`	Basis für Fehler der Service-/Business-Schicht
`exception/ResourceNotFoundException.java`	erbt jetzt von `ServiceException`
`exception/RepositoryException.java`	Persistenzschicht
`exception/MessageQueueException.java`	Messaging-Schicht
`service/Persistence.java`	übersetzt Springs `DataAccessException` in `RepositoryException`
`exception/GlobalExceptionHandler.java`	macht aus allen Exceptions die HTTP-Antworten
`ocr-worker/.../OcrProcessingException.java`	Worker-Schicht

WIE ES FUNKTIONIERT

- Ist RabbitMQ nicht erreichbar, wirft der Producer eine `MessageQueueException`. Weil `create()` `@Transactional` ist, wird das Speichern **zurückgerollt**: Es entsteht kein Dokument ohne OCR-Job. Die UI bekommt 503 mit verständlicher Meldung.
- Der Client bekommt nie interne Details (Stacktrace, SQL); die stehen nur im Log.
- Im Worker: Ein ungültiger Job wirft `OcrProcessingException`. Der Listener macht daraus `AmqpRejectAndDontRequeueException`, und die Nachricht geht in die DLQ. Zusätzlich ist `default-requeue-rejected=false` gesetzt, damit eine kaputte Nachricht nie endlos neu zugestellt wird.

Welche Exception zu welcher Antwort führt

Schicht	Exception	Ergebnis
Controller	Validierung, kaputtes JSON, ID keine Zahl	400 Bad Request
Service	`ResourceNotFoundException` (ist eine `ServiceException`)	404 Not Found
Persistenz	`RepositoryException`	500, „A database error occurred …“
Messaging	`MessageQueueException`	503, „could not be queued for OCR …“
Unerwartet	jede andere `Exception`	500, „An unexpected error occurred“
Worker	`OcrProcessingException`	Nachricht → Dead-Letter-Queue

5 Logging an wichtigen Stellen

Anforderung: Logging in remarkable/critical positions integrated

WAS

SLF4J-Logger (`LoggerFactory.getLogger(...)`) an allen Stellen, an denen etwas Wichtiges passiert oder schiefgehen kann. Das Level ist in `application.properties` einstellbar.

WO

Datei	Inhalt
`controller/*Controller.java`	eingehende Requests
`service/*Service.java`	erfolgreiche Änderungen
`messaging/OcrMessageProducer.java`	Senden an die Queue
`exception/GlobalExceptionHandler.java`	alle Fehler
`ocr-worker/.../OcrMessageListener.java`	`OcrService.java`: Verarbeitung im Worker

WIE ES FUNKTIONIERT

- Fehler werden genau **einmal mit Stacktrace** geloggt, im GlobalExceptionHandler, damit das Log nicht doppelt voll ist.
- Für mehr Details: `logging.level.at.fhtw.swen3=DEBUG` zeigt auch die Lese-Requests.

Wo	Was	Level
Controller	POST / PUT / DELETE-Requests	INFO
Controller	GET-Requests	DEBUG
Services	gespeichert, geändert, gelöscht	INFO
Producer	Job gesendet / Senden fehlgeschlagen	INFO / ERROR
Exception-Handler	4xx-Fehler / 5xx-Fehler (mit Stacktrace)	WARN / ERROR
Worker	Job empfangen, OCR gestartet, fertig / abgelehnt	INFO / ERROR

So sieht ein Upload in den Logs aus

Backend (`docker logs swen3-backend`):

```
INFO  DocumentController : POST /api/documents (filename 'rechnung-10.pdf')
INFO  DocumentService    : Document 6 saved (filename 'rechnung-10.pdf')
INFO  OcrMessageProducer : OCR job for document 6 sent to queue 'paperless.ocr'
```

OCR-Worker (`docker logs swen3-ocr-worker`), wenige Millisekunden später:

```
INFO  OcrMessageListener : Received OCR job: OcrMessage[documentId=6, filename=rechnung-10.pdf, ...]
INFO  OcrService         : Starting OCR for document 6 ('rechnung-10.pdf', application/pdf)
INFO  OcrService         : OCR is not implemented yet, skipping text recognition for document 6
INFO  OcrService         : OCR job for document 6 finished
```

Fehlerfälle selbst ausprobieren

A) RabbitMQ ist weg → 503, nichts gespeichert

```
docker stop swen3-rabbitmq
# in der UI ein Dokument anlegen → Fehlermeldung „could not be queued …“, Dokument erscheint nicht
docker start swen3-rabbitmq   # Worker verbindet sich automatisch wieder
```

B) Ungültige Nachricht → Dead-Letter-Queue

In der Management-UI: Queues → `paperless.ocr` → Publish message, Property `content_type = application/json`, Payload:

```
{"documentId": null, "filename": "x.pdf"}
```

Der Worker loggt `OCR job rejected, moving it to the dead-letter queue`, und in `paperless.ocr.dlq` liegt danach 1 Nachricht. (Mit Get messages kann man sie ansehen.)

C) Ungültige Requests → 400

```
curl -X POST localhost:8080/api/documents -H "Content-Type: application/json" -d "{ kaputt"
curl localhost:8080/api/documents/abc
```

Automatische Tests

Projekt	Neu / geändert	Ergebnis
Backend	`OcrMessageProducerTest` (senden, Fehler → MessageQueueException); `DocumentServiceTest` (+3: sendet Job, Queue weg, DB-Fehler → kein Job); `DocumentControllerTest` (+4: 503, 500 ohne Details, kaputtes JSON, ID keine Zahl)	35 Tests grün
OCR-Worker	`OcrServiceTest`, `OcrMessageListenerTest`, `OcrWorkerApplicationTests`	6 Tests grün

```
./mvnw test                    # Backend (im Projektordner)
cd ocr-worker && ./mvnw test   # Worker
```

Die Tests brauchen kein laufendes RabbitMQ: RabbitTemplate und Services werden gemockt, und im Context-Test des Workers ist der Listener abgeschaltet.

Alle Dateien auf einen Blick

Datei	Status
`docker-compose.yml`	geändert: rabbitmq, ocr-worker, RabbitMQ-Env fürs Backend
`.dockerignore`	neu: Backend-Build schickt ui-react/ und ocr-worker/ nicht mit
`pom.xml`	geändert: spring-boot-starter-amqp

Datei	Status
`src/main/resources/application.properties`	geändert: RabbitMQ, Queue-Namen, Logging
`config/RabbitMqConfig.java`	neu
`messaging/OcrMessage.java, OcrMessageProducer.java`	neu
`exception/ServiceException, RepositoryException, MessageQueueException`	neu
`exception/ResourceNotFoundException.java`	geändert: erbt von ServiceException
`exception/GlobalExceptionHandler.java`	geändert: neue Handler, Logging
`service/Persistence.java`	neu
`service/DocumentService.java, CommentService.java`	geändert: Producer, Persistence, Logging
`controller/DocumentController.java, CommentController.java`	geändert: Logging
`ocr-worker/ (pom, Dockerfile, mvnw, src/)`	neu: kompletter Worker-Dienst
`src/test/...`	geändert/neu: siehe Tests

Gut zu wissen

- **Port 8080 belegt?** Dann startet das Backend nicht („port is already allocated“). Anderes Projekt stoppen oder in der docker-compose `"8081:8080"` eintragen. Die UI auf Port 80 braucht das Mapping nicht.
- **Code geändert, aber alte Version läuft?** Immer `docker compose up -d --build`.
- **Queue-Konfiguration ändern:** Immer in beiden `RabbitMqConfig.java` (Backend und Worker) gleich ändern. Bei Problemen die alte Queue in der Management-UI löschen.
- **Nächste Schritte (spätere Sprints):** echter Datei-Upload (z. B. MinIO), echte OCR im Worker (z. B. Tesseract), Ergebnis zurück ans Backend schicken.